SPRING DOCS

Spring Data JPA

Entities, Repositories, JPQL & Transaction Management

Java Agentic AI — Knowledge Base Reference

1. JPA, Hibernate, and Spring Data JPA

Layer	Role
JPA	A specification (jakarta.persistence) defining how Java objects map to relational tables — ORM standar
Hibernate	The most widely used implementation of the JPA specification
Spring Data JPA	An abstraction on top of JPA that eliminates boilerplate DAO code via auto-generated repository implen

2. Entity Mapping

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;

    @Column(unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
    private Role role;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdAt;

    // getters, setters, constructors
}
```

GenerationType	Meaning
IDENTITY	Delegates to the DB's auto-increment column
SEQUENCE	Uses a DB sequence object (efficient, allows pre-allocation)
AUTO	Provider (Hibernate) picks the strategy based on the DB dialect
TABLE	Uses a dedicated table to simulate a sequence — portable but slower

3. Relationships

```
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;
```

```
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
}

@Entity
public class Student {
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}
```

Annotation	Default fetch type	Meaning
@OneToOne	EAGER	One entity maps to exactly one related entity
@OneToMany	LAZY	One entity owns a collection of related entities
@ManyToOne	EAGER	Many entities point to one shared entity (the 'owning' side, holds the FK)
@ManyToMany	LAZY	Many-to-many via a join table

> ***Note:*** *Best practice: override the *ToOne defaults to LAZY as well, since EAGER fetching can silently cause the N+1 query problem or load huge unnecessary object graphs.*

4. Repositories

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Derived query methods - Spring generates the implementation from the method name
    Optional<User> findByEmail(String email);
    List<User> findByRoleAndActiveTrue(Role role);
    List<User> findByNameContainingIgnoreCase(String name);
    long countByRole(Role role);
    boolean existsByEmail(String email);

    // JPQL custom query
    @Query("SELECT u FROM User u WHERE u.createdAt > :date")
    List<User> findRecentUsers(@Param("date") Date date);

    // Native SQL query
    @Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = true)
    User findByEmailNative(String email);

    // Modifying query
    @Modifying
    @Query("UPDATE User u SET u.active = false WHERE u.id = :id")
    void deactivateUser(@Param("id") Long id);
}
```

JpaRepository provides save(), findById(), findAll(), delete(), count() out of the box — Spring Data generates a proxy implementation at startup by parsing the interface, no manual DAO class needed.

5. Pagination and Sorting

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("name").ascending());
Page<User> page = userRepository.findAll(pageable);

page.getContent();        // the actual list of results
page.getTotalElements();
page.getTotalPages();
page.hasNext();
```

6. Transactions

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        inventoryService.reduceStock(order.getItems()); // if this throws, the save() above
rolls back too
    }

    @Transactional(readOnly = true)     // optimization hint - no dirty checking, can use
read replicas
    public List<Order> getOrders() {
        return orderRepository.findAll();
    }
}
```

Propagation	Behavior
REQUIRED (default)	Join the existing transaction, or start a new one if none exists
REQUIRES_NEW	Always suspend any existing transaction and start a fresh independent one
NESTED	Start a nested transaction (savepoint) within the existing one, if supported
MANDATORY	Must run within an existing transaction, or throw an exception

Note: *@Transactional works via a Spring AOP proxy around the bean — calling a @Transactional method from ANOTHER method in the SAME class bypasses the proxy entirely, silently skipping the transaction. This 'self-invocation' pitfall is a classic Spring interview trap.*

7. The N+1 Query Problem

Occurs when fetching a list of N parent entities triggers 1 query for the parents plus N additional queries — one per parent — to lazily fetch each one's related collection, instead of a single efficient join.

```
// Problem: 1 query for orders, then N queries for order.getUser() per order
List<Order> orders = orderRepository.findAll();
orders.forEach(o -> System.out.println(o.getUser().getName())); // N+1!

// Fix 1: JOIN FETCH in JPQL
```

```
@Query("SELECT o FROM Order o JOIN FETCH o.user")
List<Order> findAllWithUser();

// Fix 2: @EntityGraph
@EntityGraph(attributePaths = {"user"})
List<Order> findAll();
```

8. Quick Interview Recap

- JPA vs Hibernate? JPA is a specification/interface; Hibernate is a concrete implementation of it (the most common one).
- What is the N+1 problem and how do you fix it? Lazy-loading a collection per parent row triggers one extra query per row; fix with JOIN FETCH, @EntityGraph, or batch fetching.
- Why does self-invocation break @Transactional? Spring's declarative transaction management relies on an AOP proxy wrapping the bean; calling a method directly via 'this' inside the same class bypasses that proxy, so the annotation is never intercepted.
- Default fetch type for @OneToMany vs @ManyToOne? @OneToMany defaults to LAZY; @ManyToOne and @OneToOne default to EAGER (often overridden to LAZY in practice).
- save() vs saveAndFlush()? save() may batch the write until the persistence context flushes naturally; saveAndFlush() forces an immediate synchronization with the database.